

TEAM OCEANUS

Texas A&M · MATE ROV · Guidance, Navigation & Control

6-DoF Simulation & Servo Control

ArduSub SITL — Work Summary (Goals 1 & 2)

What this covers

- Goal 1 — a custom 6-DoF (vectored_6dof / 8-thruster) frame configured and validated in Software-In-The-Loop against the real ArduSub firmware.
- Goal 2 — the manipulator servo control architecture, with a latency and precision test report.
- Bonus — water-current disturbance modelling and a tuned PID station-keeping loop for ConOps current compensation.
- A reproducible Docker-based kit, telemetry logging, and shareable dashboards.

Prepared by GNC subteam

June 2026 · ArduSub V4.5.7 · macOS (Apple Silicon) + Docker

Overview & status

The GNC 3-week plan called for finalizing the control configuration, validating it in simulation, validating the servo control architecture, and refining the photogrammetry pipeline. This document summarizes the simulation and servo work (Goals 1 & 2).

Approach: rather than a hand-rolled physics sim, we run the real ArduSub firmware in Software-In-The-Loop (SITL) inside Docker. The same frame mixing, PID loops, and MAVLink commands tested here behave identically on the Navigator + Raspberry Pi 5 + BlueOS flight stack — so this is true control-software validation, not an approximation.

Item	Status	Deliverable
Goal 1 — 6-DoF vehicle simulation	Complete	Custom frame, validated SITL, config guide
Goal 2 — Servo control architecture	Complete	Servo scripts + latency/precision report
Current compensation (ConOps)	Complete	Water-current model + tuned PID hold
Practice Project 2 — MAVLink IMU	Complete	Heartbeat + live IMU/attitude/depth
Goal 3 — Photogrammetry pipeline	Next	RTSP drop/latency logging (app repo)

Why SITL is the right tool

SITL exercises the production autopilot binary. A custom motor matrix that mixes 8 thrusters correctly in SITL will mix them correctly on hardware, and a PID tuned here ports directly. The only gap is the simulated physics body (mass, drag, buoyancy) — which we customized to approximate our ROV.

Goal 1 — 6-DoF frame configuration

The team's final design is an 8-thruster frame: 4 vectored horizontal thrusters (surge / sway / yaw) plus 4 vertical thrusters (heave / roll / pitch). In ArduSub this is the vectored_6dof frame, FRAME_CONFIG = 2 (BlueROV2 Heavy geometry). The 4 vertical thrusters are what make roll and pitch independently controllable — a 2-vertical-thruster frame cannot do this, so it is the reason 6-DoF requires 8 thrusters.

Validation — every DoF drives the expected thrusters

Driving each degree of freedom in turn and reading SERVO_OUTPUT_RAW confirms the mix. Independent roll and pitch (vertical group only) prove true 6-DoF:

Command	Horizontal 1-4 (μ s)	Vertical 5-8 (μ s)
neutral	1500 1500 1500 1500	1500 1500 1500 1500
forward	1250 1250 1750 1750	1500 1500 1500 1500
lateral	1750 1250 1750 1250	1500 1500 1500 1500
heave	1500 1500 1500 1500	1250 1250 1250 1250
yaw	1750 1250 1250 1750	1500 1500 1500 1500
roll	1500 1500 1500 1500	1750 1250 1750 1250
pitch	1500 1500 1500 1500	1250 1250 1750 1750

Deliverables: a custom-frame parameter set (params/vectored_6dof.parm, also loadable onto the real Navigator), the validated Docker SITL environment, and a step-by-step custom-frame config guide in the kit README.

Goal 1 — Oceanus vehicle physics model

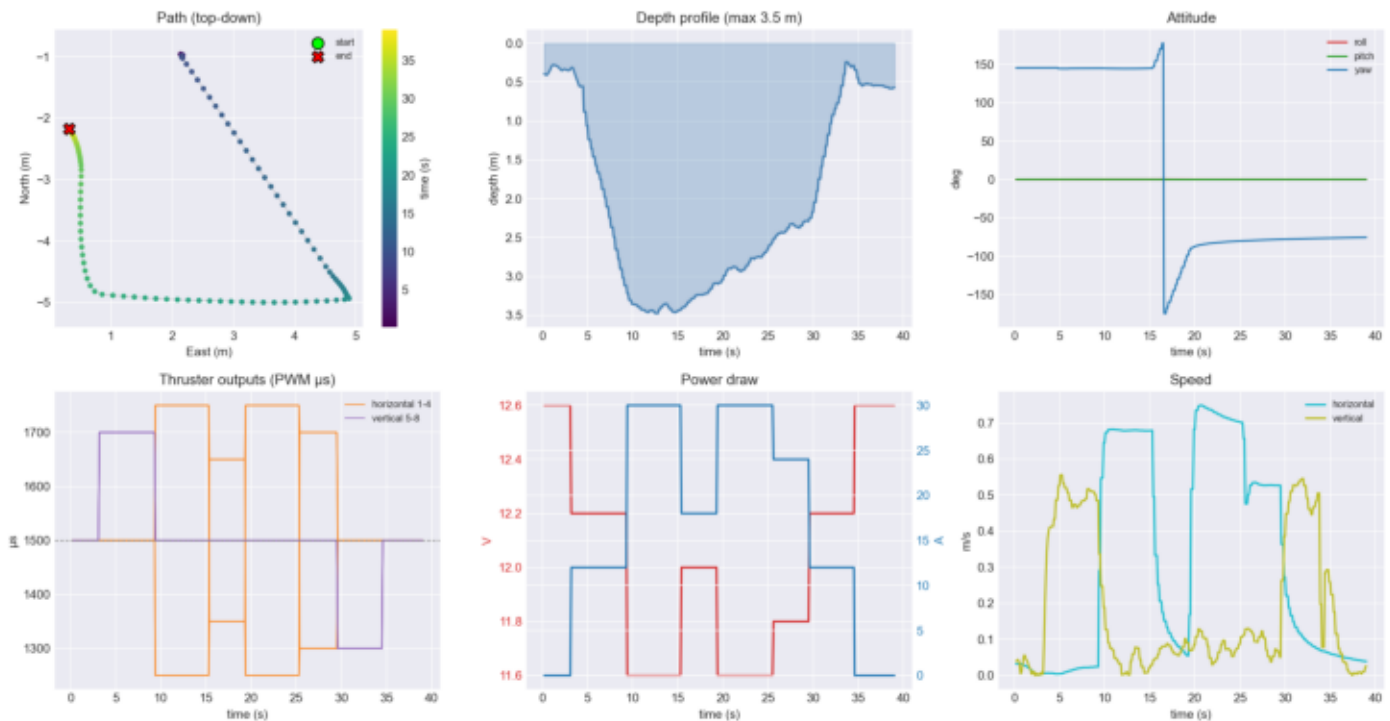
ArduSub SITL defaults to ArduPilot's generic BlueROV2 body. We patch the SITL physics (build-time) with Team Oceanus numbers so the simulated dynamics — how it accelerates, glides, and turns — approximate our ROV. The control firmware is always real; only the physics body is substituted. The TD1.2 thrusters (~24.5 N) are well under half a T200, so the modelled craft is heavier and gentler than a stock BlueROV2.

Parameter	Value	Confidence
Thrusters	8 × Diamond Dynamics TD1.2	real
Thrust per thruster	24.5 N (~2.5 kgf max)	real
Frame	vectored_6dof (FRAME_CONFIG 2)	real
Mass	22.0 kg	design target
Dimensions (L×W×H)	0.60 × 0.50 × 0.35 m	estimate
Buoyancy	~neutral	target
Drag coefficients	ArduPilot defaults	unchanged

Goal 1 — telemetry & visualization

Every run is logged to CSV (depth, attitude, NED position, velocity, power, all 8 thruster outputs) and rendered into one shareable dashboard — so testing reads as a picture, not a wall of numbers. Below: the Oceanus-model vehicle flying the standard maneuver.

ROV SITL — Oceanus (22kg, Diamond Dynamics TD1.2 24.5N) (39s run)

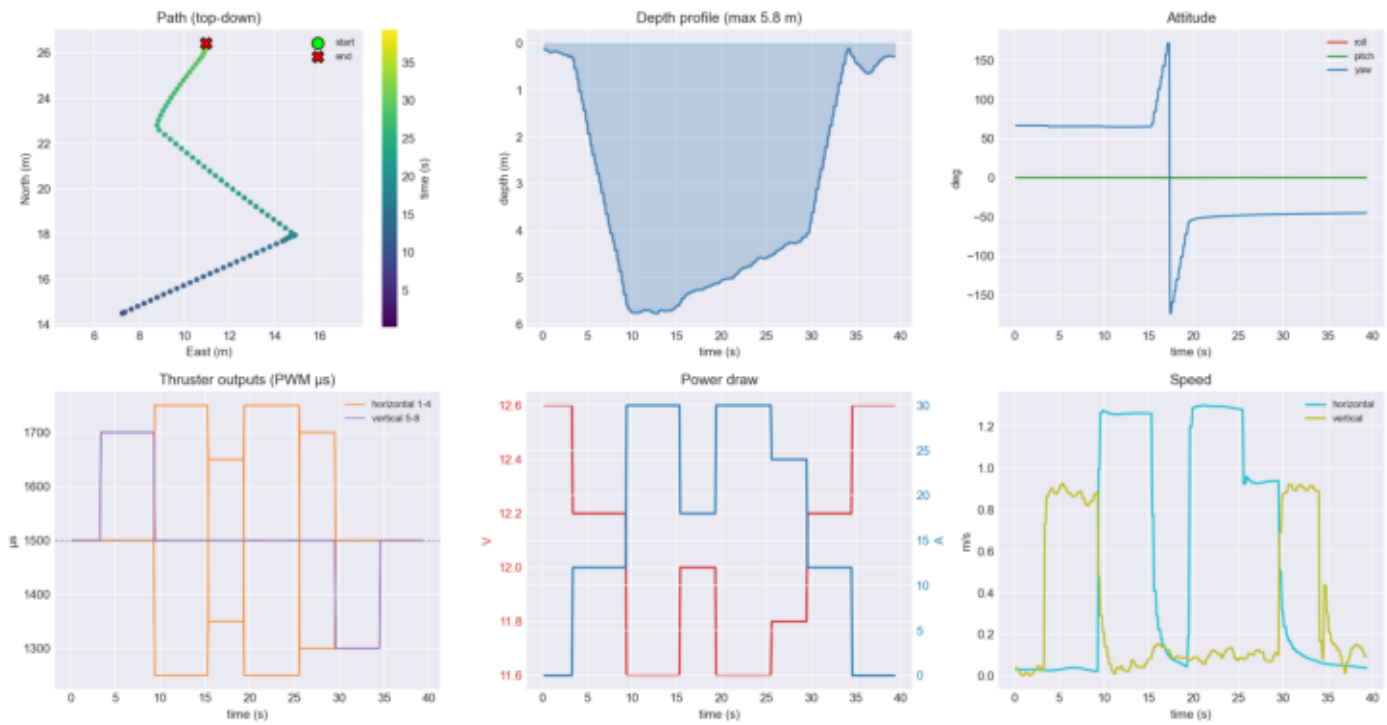


Oceanus model (22 kg, TD1.2): scripted dive → cruise → turn → strafe → surface.

Goal 1 — baseline comparison

The same commands on the stock BlueROV2 reference vehicle dive deeper and faster — confirming the Oceanus model's heavier mass and weaker TD1.2 thrust are reflected in the simulated dynamics.

ROV SITL Test — Full Maneuver (39s run)

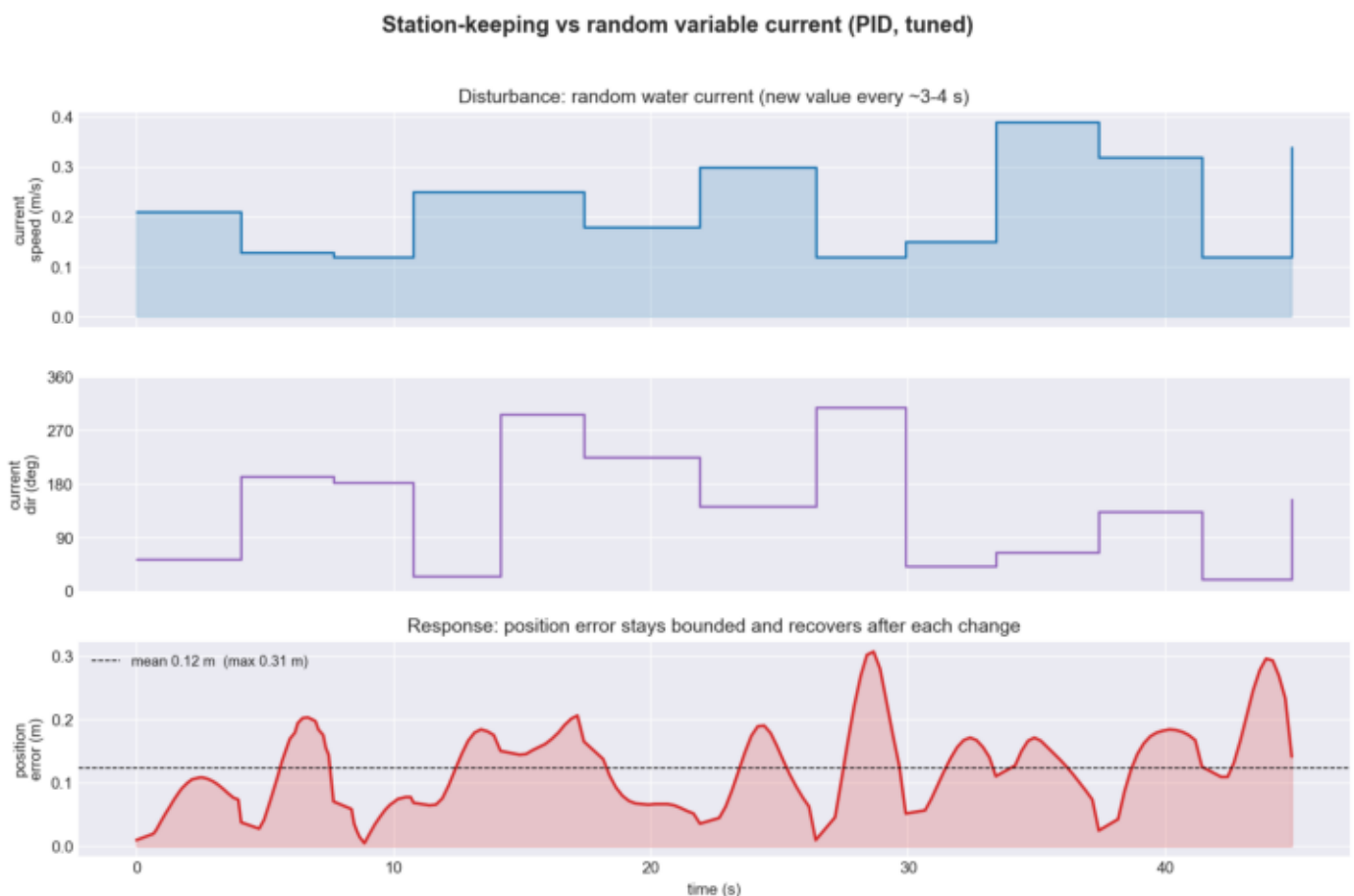


Stock BlueROV2 (10.5 kg, T200-class) flying the identical command sequence.

Current disturbance & station-keeping

Stock SITL is still water, so we patched the model so the wind parameters act as a horizontal water current — letting us test the ConOps drift / current-compensation task. Unpowered, the ROV drifts with the current; a PID position-hold loop (NED error → forward/lateral thrust, depth held by the autopilot) cancels it.

After tuning (Kp 440 / Kd 230 / Ki 210, with integral anti-windup removing the steady-state droop), it holds to ~ 0.01 m against a constant 0.4 m/s current and mean ~ 0.12 m against a randomly varying one. The trace below overlays the disturbance and the resulting position error.



Tuned PID station-keeping vs a random current (13 changes): mean 0.12 m, max 0.31 m.

Goal 2 — servo control architecture

The manipulator servos sit on PWM channels 14–16 (GNC-ICD-01) and default to `SERVOn_FUNCTION = 0` (Disabled), which lets the GCS drive them directly via `MAV_CMD_DO_SET_SERVO` — no reboot or reconfig. `servo_control.py` exposes raw-PWM, position, angle, gripper, and sweep commands over that same MAVLink path that runs on the Navigator.

`servo_test.py` is the deliverable test report: command-path latency ~10 ms mean (p95 ~18 ms) and 0 μ s command error in SITL. SITL validates the software/comms architecture; mechanical travel time and physical positional accuracy are flagged as bench tests to add on hardware.



Servo command path on channel 14: latency histogram (left), commanded vs actual PWM (right).

How to reproduce & artifact index

Run it

- `cd 01_Subteams/GNC/SITL-Simulation && ./run_sitl.sh` (builds the ArduSub SITL Docker image, then launches)
- `Scripts/pymavlink connect on tcp:127.0.0.1:5780; QGroundControl on tcp:localhost:5781.`
- `python3 -m venv .venv && pip install -r scripts/requirements.txt`

Key scripts

- `sitl_check.py` — heartbeat + live IMU/attitude/depth (Project 2)
- `thruster_mixing.py` — drive each DoF, confirm 8-thruster mixing
- `log_telemetry.py` / `plot_telemetry.py` — record a run, render a dashboard PNG
- `inject_current.py` / `station_keeping.py` — water current + PID compensation
- `servo_control.py` / `servo_test.py` — manipulator servos + Goal 2 test report

Documentation

- `README.md` — full kit docs + step-by-step custom-frame config guide
- `OCEANUS-VEHICLE-MODEL.md` — physics values, sources, how to update
- `Frame-Config-Decision.md` — the 8-thruster / vectored_6dof decision
- `sample-runs/` — every CSV + dashboard referenced in this document

Caveats / next

- The vehicle mass & dimensions are design targets — regenerate the runs once the assembled ROV is weighed.
- Goal 3 (photogrammetry RTSP drop/latency logging) is the remaining 3-week plan item, in the `akvaithi/MATE-ROV-Photogrammetry` app repo.